

Rust

Le langage · possession et emprunt · types et traits · erreurs · collections · Cargo · concurrence

let **x** = **5**; chasse fixe, gras : code à taper tel quel
nom italique : valeur à remplacer
// → 42 commentaire : résultat obtenu
E0382 code d'erreur du compilateur
❶ piège fréquent ou action risquée

INSTALLER ET DÉMARRER

rustup	installateur officiel et gestionnaire de versions (rustup.rs)
rustup update	mettre à jour compilateur et outils
cargo new projet	créer un projet (--lib : une bibliothèque)
cargo run	compiler et lancer
cargo check	vérifier sans produire d'exécutable (rapide)
cargo build --release	compilation optimisée → target/release/
cargo test	lancer les tests
cargo fmt	mettre le code en forme
cargo clippy	conseils de qualité (analyseur)
rustup doc --std	documentation de la bibliothèque standard, hors ligne

Windows : **rustup-init.exe** ; il propose d'installer les outils Visual Studio (éditeur de liens MSVC) requis.

Une version stable toutes les 6 semaines ; au 29 septembre 2026 : **1.98**. Édition courante : **2024** (depuis Rust 1.85).

PREMIER PROGRAMME

```
// src/main.rs
fn main() {
    let nom = "monde";
    println!("Bonjour, {nom} !");
}
```

- fn main()** : point d'entrée du programme.
- println!** : le ! signale une **macro**.
- Instruction terminée par ; ; blocs entre { }.
- Commentaires : // ligne · /* */ bloc · /// documentation de l'élément qui suit.
- Noms : **snake_case** variables, fonctions · **CamelCase** types, traits · **MAJUSCULES** constantes.

VARIABLES

let x = 5;	liaison immuable (par défaut)
let mut n = 0;	liaison modifiable
let x: i64 = 5;	type explicite (sinon déduit)
let x = x * 2;	masquage : nouvelle variable de même nom, type libre
const MAX: u32 = 100;	constante : type obligatoire, calculée à la compilation
static NOM: &str = "a";	valeur globale unique
let (a, b) = (1, 2);	déstructuration
let _x = 1;	_ en tête : pas d'avertissement si inutilisée

❶ Réaffecter une liaison sans **mut** : erreur **E0384**.

TYPES SCALAIRES

i8 i16 i32 i64 i128	entiers signés ; i32 par défaut
u8 u16 u32 u64 u128	entiers non signés ; u8 = octet
isize usize	taille d'une adresse (64 bits sur un PC actuel) ; index et longueurs : usize
f32 f64	flottants IEEE 754 ; f64 par défaut
bool	true · false
char	un caractère Unicode, 4 octets : 'é'
()	« unité » : absence de valeur

LITTÉRAUX

1_000_000	_ : séparateur visuel
0xff 0o77 0b1010	hexadécimal · octal · binaire
255u8 2.5f32	suffixe de type
b'A' b"abc"	octet u8 · chaîne d'octets
'\n' '\t' '\\'	caractères échappés
'\u{e9}'	(\u{e9} = é)
r"C:d" · r#"a "b"#"	chaînes brutes : \ et " pris à la lettre

Bornes : **i32::MAX**, **u8::MIN**... ❶ Dépassement d'entier : panique en mode debug, résultat « rebouclé » en **--release**. Choisir : **checked_add** (→ **Option**), **wrapping_add**, **saturating_add**.

TYPES COMPOSÉS

(i32, f64, char)	tuple : types mêlés, taille fixe ; t.0 , t.1
[i32; 5]	tableau : taille fixe ; [0; 5] = cinq zéros
&[i32]	tranche (slice) : vue sur une suite contiguë ; &a[1..3]
&str	tranche de texte UTF-8 ; littéraux "..."
String	texte possédé, modifiable (tas)
Vec<T>	tableau dynamique possédé (tas)

a[i] hors limites : **panique**, jamais de lecture hasardeuse. **a.get(i)** renvoie une **Option**.

CONVERSIONS

x as f64	entre types primitifs, sans contrôle
f64::from(x) · x.into()	sans perte possible (From / Into)
u8::try_from(x)	faillible → Result
"42".parse::<i32>()	texte → nombre : Result
n.to_string()	nombre → String
s.as_str() · &s	String → &str
s.to_string()	&str → String (ou String::from(s))

❶ **as** tronque : **300_i32 as u8** → 44 ; **-1_i32 as u32** → 4294967295 ; **2.9_f64 as i32** → 2 ; **300.0_f64 as u8** → 255.

AFFICHER : PRINTLN! ET FORMAT!

{}	affichage « utilisateur » (Display)
{:?}", {:#?}	débogage (Debug) · indenté
{x} · {0} · {nom}	variable · position · argument nommé
{:>8} {:<8} {:^8}	à droite · à gauche · centré sur 8
{:08.2}	8 caractères, zéros devant, 2 décimales
{:+}	signe toujours affiché
{:x} {:X} {:o} {:b}	hexadécimal · majuscules · octal · binaire
{:#x} · {:#b}	avec préfixe 0x · 0b
{:e}	notation scientifique
{:>w\$} · {:.p\$}	largeur · précision prises dans w · p
{{ · }}	accolade littérale

MACROS D'USAGE COURANT

print! · println!	sortie standard
eprint! · eprintln!	sortie d'erreur
format!(...)	même syntaxe → String
dbg!(expr)	affiche fichier, ligne, expression et valeur (sur stderr), puis la renvoie
write! · writeln!	vers un fichier ou un tampon
vec![1, 2, 3]	construire un Vec
assert!(c) · assert_eq!(a, b)	vérifier, sinon paniquer
todo!()	code à écrire : panique si atteint
unreachable!()	branche jugée impossible

LIRE LE COMPILATEUR

- Lire la **première** erreur : les suivantes en découlent souvent.
- Code **E0382** → **rustc --explain E0382** : explication et exemples.
- Suivre les lignes **help** : le correctif y est souvent proposé.
- cargo check** pour itérer vite ; **cargo clippy** pour les tournures idiomatiques.

E0308	types incompatibles	conversion ? ; final en trop ?
E0277	trait non implémenté	derive , impl , autre type
E0599	méthode introuvable	type exact ? trait importé ?
E0425	nom inconnu	orthographe, portée
E0432 · E0433	import · chemin non résolu	use , mod , dépendance

Erreurs d'emprunt : page 3. Hors blocs **unsafe**, le compilateur écarte des classes entières d'erreurs mémoire : pointeur pendant, double libération, accès concurrent non synchronisé.

2 · Opérateurs, fonctions, contrôle du flux

MÉMENTO — RUST

OPÉRATEURS

<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>	arithmétique ; entre entiers, / division entière : <code>7 / 2</code> → 3, % reste
<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code>	comparaison
<code>&&</code> <code> </code> <code>!</code>	logique (évaluation paresseuse)
<code>&</code> <code> </code> <code>^</code> <code><<</code> <code>>></code>	bit à bit ; ! <code>n</code> : complément
<code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code>	affectation composée
<code>a..b</code> <code>·</code> <code>a..=b</code>	intervalle exclusif · inclusif
<code>&x</code> <code>·</code> <code>&mut x</code> <code>·</code> <code>*r</code>	emprunter · en modification · déréférencer
<code>?</code>	propager une erreur (page 5)
<code>a::b</code>	chemin : <code>std::fs::read</code> , <code>String::new</code>

Ni `++` ni `--` : `n += 1`. Puissance : `2_i32.pow(10)`, `x.powf(0.5)`, `x.sqrt()`.
Arrondis : `round`, `floor`, `ceil` · bornes : `min`, `max`, `clamp` · reste positif :
`rem_euclid` (`-7 % 3` → -1). ⚠ Flottants : comparer `(a - b).abs() < 1e-9`, pas `a == b` après un calcul.

TOUT EST EXPRESSION

```
let signe = if n < 0 { "négatif" } else { "positif" };

let carre = {
    let a = 3;
    a * a           // pas de ; : valeur du bloc
};                // carre = 9
```

Un bloc vaut sa dernière expression **sans** `;` ; avec `;`, il vaut `()`. Les branches d'un `if` doivent avoir le même type.

FONCTIONS

```
fn aire(l: f64, h: f64) -> f64 {
    l * h                       // valeur renvoyée
}

fn diviser(a: i32, b: i32) -> Option<i32> {
    if b == 0 {
        return None;           // sortie anticipée
    }
    Some(a / b)
}

fn saluer(nom: &str) {          // renvoie ()
    println!("Bonjour {nom}");
}
```

- Types des paramètres **obligatoires** ; type de retour après `->`.
- Ni paramètres par défaut ni surcharge : **Option**, noms distincts, constructeurs.
- ⚠ `fn f() -> i32 { 5; }` : le `;` fait renvoyer `()` → **E0308**.
- Retour `-> !` : la fonction ne revient jamais (**panic!**, `std::process::exit`).

FERMETURES (CLOSURES)

```
let double = |x| x * 2;
let somme = |a: i32, b: i32| -> i32 { a + b };
let seuil = 10;
let grand = |x: &i32| *x > seuil; // capture seuil
let f = move || println!("{seuil}"); // prend possession
```

Fn	lit ce qu'elle capture ; callable à volonté
FnMut	modifie ce qu'elle capture
FnOnce	consomme ce qu'elle capture : un seul appel
fn(i32) -> i32	pointeur de fonction (aucune capture)

`move` : indispensable quand la fermeture survit à la portée courante (thread, valeur renvoyée). Accepter une fermeture :

```
fn appliquer(f: impl Fn(i32) -> i32).
```

BOUCLES

```
for i in 0..5 { }                // 0, 1, 2, 3, 4
for i in 1..=5 { }               // 1 à 5 inclus
for i in (0..10).rev() { }       // 9 à 0
for i in (0..10).step_by(3) { }  // 0, 3, 6, 9
for (i, x) in v.iter().enumerate() { }

while n > 0 { n -= 1; }

let mut k = 0;
let r = loop {                  // boucle sans fin...
    k += 1;
    if k == 10 { break k * 2; }  // ...qui renvoie 20
};

'externe: for x in 0..5 {        // étiquette
    for y in 0..5 {
        if x + y > 6 { break 'externe; }
        if y > x { continue 'externe; }
    }
}
```

for `x` in `&v` parcourt sans consommer ; **for** `x` in `v` consomme `v` (inutilisable ensuite) ; **for** `x` in `&mut v` permet de modifier : `*x += 1`.

MATCH : FILTRAGE PAR MOTIF

```
match n {
    0 => println!("zéro"),
    1 | 2 => println!("un ou deux"),
    3..=9 => println!("chiffre"),
    x if x < 0 => println!("négatif : {x}"),
    grand @ 100.. => println!("grand : {grand}"),
    _ => println!("autre"),
}

let pays = match code {         // une expression
    'F' => "France",
    'D' => "Allemagne",
    _ => "inconnu",
};
```

<code>_</code>	n'importe quoi, sans lier
<code>x</code>	lie la valeur à <code>x</code>
<code>a b</code>	l'un ou l'autre
<code>1..=5</code> <code>·</code> <code>'a'..='z'</code>	intervalle inclusif
<code>x if condition</code>	garde : condition supplémentaire
<code>n @ 1..=9</code>	tester et lier à la fois
<code>(a, _, c)</code>	tuple
<code>[premier, .., dernier]</code>	tableau ou tranche ; <code>..</code> : le reste
<code>Point { x, y: 0, .. }</code>	structure : champs lus, champs testés
<code>Some(v) · Err(e)</code>	variante d'une énumération
<code>&x</code>	déstructurer une référence

⚠ Un **match** doit être **exhaustif** : tout cas possible couvert, sinon **E0004**. `_` en dernier recueille le reste.

RACCOURCIS DE FILTRAGE

```
if let Some(x) = option {        // un seul motif
    println!("{x}");
} else {
    println!("rien");
}

let Some(x) = option else {      // let ... else
    return;                     // doit sortir
};                               // x reste utilisable

while let Some(sommet) = pile.pop() {
    println!("{sommet}");
}

if let Some(x) = option && x > 10 { } // édition 2024

let voyelle = matches!(c, 'a' | 'e' | 'i' | 'o' | 'u');
```

Le bloc **else** d'un **let ... else** doit diverger : **return**, **break**, **continue**, **panic!**. Chaînes **if let ... &&** : édition 2024, Rust 1.88 ou plus.

3 · Possession, emprunt, durées de vie

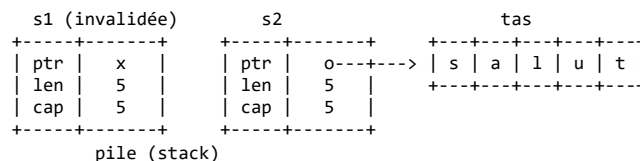
MÉMENTO — RUST

LES TROIS RÈGLES DE LA POSSESSION (OWNERSHIP)

- Chaque valeur a un **seul propriétaire** : une variable, un champ, un élément de collection.
- Affecter la valeur ou la passer à une fonction la **déplace** (move) : l'ancien nom devient inutilisable.
- Quand le propriétaire sort de sa portée, la valeur est **libérée** (drop), automatiquement : ni ramasse-miettes ni **free**.

Les types **Copy** (entiers, flottants, **bool**, **char**, **&T**, tuples et tableaux de types **Copy**) sont **copiés** au lieu d'être déplacés. Les autres (**String**, **Vec**, **Box**...) se dupliquent explicitement avec **.clone()** : une copie coûteuse reste visible.

```
let s1 = String::from("salut");
let s2 = s1;           // déplacement : s1 n'est plus utilisable
```



DÉPLACER, COPIER, CLONER

```
let s1 = String::from("salut");
let s2 = s1;           // déplacement
// println!("{}", s1); // E0382 : valeur déplacée

let s3 = s2.clone();   // copie profonde
println!("{}", s2);    // Les deux sont valides

let a = 5;
let b = a;             // i32 est Copy
println!("{}", b);     // a reste valide

fn prendre(s: String) { } // prend possession
prendre(s3);           // s3 inutilisable ensuite
```

CHOISIR LE TYPE D'UN PARAMÈTRE

&str	lire un texte (accepte &String et les littéraux)
&[T]	lire une suite (accepte &Vec<T> et les tableaux)
&mut T	modifier sur place
T	garder la valeur (la stocker) ; petites valeurs Copy

Renvoyer une valeur **possédée** (**String**, **Vec<T>**) plutôt qu'une référence vers une variable locale.

EMPRUNTER : LES RÉFÉRENCES

```
fn longueur(s: &String) -> usize { s.len() } // Lit
fn crier(s: &mut String) { s.push('!'); }     // modifie

let mut s = String::from("salut");
let n = longueur(&s);           // emprunt partagé
crier(&mut s);                 // emprunt exclusif
```

Règle d'or — à un instant donné : **soit** autant de références partagées **&T** qu'on veut, **soit** une seule référence exclusive **&mut T**. Et une référence ne vit jamais plus longtemps que la valeur qu'elle désigne.

```
let mut v = vec![1, 2, 3];
let premier = &v[0];           // emprunt partagé...
v.push(4);                     // E0502 : exclusif
println!("{}", premier);       // ...encore utilisé ici
```

Pourquoi ? **push** peut réallouer le vecteur : **premier** désignerait une mémoire libérée. Correctifs : utiliser **premier** avant le **push**, ou copier la valeur (**let premier = v[0];**).

Un emprunt dure jusqu'à sa **dernière utilisation**, pas jusqu'à la fin du bloc.

DURÉES DE VIE ('a)

```
fn plus_long<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}

struct Extrait<'a> {
    texte: &'a str,           // la structure emprunte
}
```

- 'a ne change aucune durée : il **décrit** le lien entre les références (ici : le résultat vit autant que la plus courte des deux).
- Élision : une seule référence en paramètre, ou **&self** → annotation inutile.
- 'static : valable pendant tout le programme (littéraux "...").

PORTÉE ET LIBÉRATION

```
{
    let a = String::from("a");
    let b = vec![1, 2, 3];
    // ... a et b utilisables ici
} // fin de portée : b, puis a, libérés
let v = vec![0; 1000];
drop(v); // libération anticipée
```

Ordre inverse de la déclaration. **impl Drop** : agir à la libération (fermer un fichier, relâcher un verrou).

POINTEURS INTELLIGENTS ET PARTAGE

Box<T>	valeur sur le tas, un seul propriétaire	type récursif, dyn Trait
Rc<T>	plusieurs propriétaires (compteur de références)	partage dans un thread
Arc<T>	idem, compteur atomique	partage entre threads
Cell<T>	modifier à travers & (mutabilité intérieure)	contrôle à l'exécution
RefCell<T>	mutabilité intérieure verrouillée	entre threads
Mutex<T>		
RwLock<T>		
Weak<T>	référence non possédante vers un Rc / Arc	casser les cycles
Cow<'a, T>	emprunté ou possédé selon le besoin	éviter une copie

Associations classiques : **Rc<RefCell<T>>** (un thread) · **Arc<Mutex<T>>** (plusieurs threads).

❗ **RefCell** : deux **borrow_mut()** simultanés → panique. Deux **Rc** qui se désignent mutuellement ne sont jamais libérés : fuite mémoire (utiliser **Weak**).

PRENDRE UNE VALEUR HORS D'UN EMPRUNT (E0507)

&v[i]	emprunter au lieu de prendre
v[i].clone()	copier l'élément (coût d'une copie)
mem::take(&mut v[i])	prendre, en laissant la valeur par défaut (" ", 0, vide)
option.take()	prendre le contenu, en laissant None
v.into_iter() · v.drain(..)	parcourir en prenant possession · en vidant

CODES D'ERREUR FRÉQUENTS

E0382	valeur utilisée après déplacement	emprunter &, cloner, réordonner
E0499	deux emprunts &mut simultanés	raccourcir le premier
E0502	&mut pendant un emprunt &	finir de lire avant d'écrire
E0505	déplacement d'une valeur empruntée	attendre la fin de l'emprunt
E0507	sortie d'une valeur depuis une référence	&, .clone(), mem::take
E0596	&mut sur une variable non mut	let mut
E0597	valeur libérée trop tôt	allonger sa portée
E0515	référence vers une variable locale renvoyée	renvoyer la valeur possédée
E0106	durée de vie manquante	annoter 'a ou posséder

Détail et exemples : **rustc --explain code** ou doc.rust-lang.org/error_codes.

4 · Structures, énumérations, traits, génériques

STRUCTURES ET MÉTHODES

```
#[derive(Debug, Clone, PartialEq)]
struct Point {
    x: f64,
    y: f64,
}

impl Point {
    fn new(x: f64, y: f64) -> Self { // fonction associée
        Self { x, y } // raccourci de x: x
    }
    fn norme(&self) -> f64 { self.x.hypot(self.y) }
    fn deplacer(&mut self, dx: f64) { self.x += dx; }
    fn eclater(self) -> (f64, f64) { (self.x, self.y) }
}

let mut p = Point::new(3.0, 4.0);
println!("{}", p.norme()); // 5
p.deplacer(1.0);
let q = Point { y: 0.0, ..p.clone() }; // autres champs de p
let Point { x, y } = q; // déstructurer
```

&self	la méthode lit l'instance
&mut self	la méthode la modifie
self	la méthode la consomme
Self	le type lui-même, dans un impl
struct Metres(f64);	structure-tuple : m.0 ; « newtype »
struct Vide;	structure unité, sans champ
type Km = f64;	alias : autre nom du même type

ÉNUMÉRATIONS

```
use std::f64::consts::PI;

enum Forme {
    Cercle { rayon: f64 },
    Rectangle(f64, f64),
    Point,
}

impl Forme {
    fn aire(&self) -> f64 {
        match self {
            Forme::Cercle { rayon } => PI * rayon * rayon,
            Forme::Rectangle(l, h) => l * h,
            Forme::Point => 0.0,
        }
    }
}
```

Une valeur d'énumération est **une** de ses variantes, chacune avec ses propres données : idéal pour modéliser des états.

Dans le prélude, sans `use` : `enum Option<T> { None, Some(T) }` et `enum Result<T, E> { Ok(T), Err(E) }`. Rust n'a pas de `null`.

TRAITS : COMPORTEMENTS PARTAGÉS

```
trait Decrire {
    fn nom(&self) -> String; // à fournir
    fn decrire(&self) -> String { // fournie par défaut
        format!("Je suis {}", self.nom())
    }
}

impl Decrire for Point {
    fn nom(&self) -> String { "un point".to_string() }
}

use std::fmt;
impl fmt::Display for Point { // active {} et to_string
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "{}", self.x, self.y)
    }
}
```

Implémenter un trait pour un type : permis si le trait **ou** le type est défini dans votre crate (règle de l'orphelin).

GÉNÉRIQUES ET CONTRAINTES

```
fn plus_grand<T: PartialOrd>(v: &[T]) -> &T {
    let mut max = &v[0];
    for x in v {
        if x > max { max = x; }
    }
    max
}

fn afficher(x: impl fmt::Display) { println!("{}", x); }

fn resumer<T>(x: &T) -> String
where
    T: Decrire + Clone,
{
    x.decrire()
}

struct Pile<T> { elements: Vec<T> }
impl<T> Pile<T> {
    fn empiler(&mut self, x: T) { self.elements.push(x); }
}
```

T: Trait	contrainte : T doit implémenter Trait
T: A + B	plusieurs contraintes
where ...	contraintes nombreuses, écrites à part
x: impl Trait	paramètre : raccourci d'un générique
-> impl Trait	renvoie « un type qui implémente Trait » (itérateur...)
&dyn Trait · Box<dyn Trait>	objet-trait : type choisi à l'exécution

Générique : une copie du code par type, à la compilation (monomorphisation) — rapide. **dyn** : un seul code, appel indirect ; permet les collections hétérogènes : `Vec<Box<dyn Decrire>>`.

TRAITS STANDARD À CONNAÎTRE

Debug	<code>{:?}</code>	derive
Display	<code>{}, .to_string()</code>	à écrire
Clone · Copy	<code>.clone()</code> · copie implicite	derive
PartialEq · Eq	<code>==, !=</code>	derive
PartialOrd · Ord	<code><, >, .sort(), .max()</code>	derive
Hash	clé de HashMap (avec Eq)	derive
Default	<code>T::default()</code>	derive
From · Into	conversions ; From fournit Into	à écrire
FromStr	<code>"...".parse()</code>	à écrire
Iterator	boucle for , adaptateurs : fournir <code>next()</code>	à écrire
Drop	code exécuté à la libération	à écrire
Deref	<code>*x</code> ; méthodes transmises (Box , String)	à écrire
Add, Mul, Neg...	surcharge des opérateurs (<code>std::ops</code>)	à écrire
Error	type d'erreur (avec Debug + Display)	à écrire
Send · Sync	transfert · partage entre threads	automatique

❗ **f64** n'implémente ni **Eq** ni **Ord** (à cause de NaN) : `v.sort_by(|a, b| a.total_cmp(b))`.

VISIBILITÉ

(rien)	privé : visible dans le module et ses sous-modules
pub	public
pub(crate)	visible dans toute la crate
pub(super)	visible dans le module parent

❗ Un **pub struct** garde ses champs privés : mettre **pub** devant chaque champ voulu.

5 · Gestion des erreurs, entrées-sorties

MÉMENTO — RUST

TROIS SITUATIONS

panic!	irré récupérable : bogue, invariant violé ; le thread s'arrête
Result<T, E>	ré récupérable : l'appelant décide (fichier absent, saisie invalide, réseau)
Option<T>	absence de valeur, sans erreur (recherche vide, champ facultatif)

Ni exceptions ni **null** : les échecs possibles se lisent dans la signature des fonctions.

QUAND PANIQUER ?

- **Oui** : bogue, invariant impossible, tests, prototypes ; **expect** sur une hypothèse démontrée.
- **Non** : tout ce qui vient de l'extérieur (fichier, saisie, réseau, données).
- Paniques implicites : index hors limites, division entière par zéro, dépassement (debug), **unwrap** sur **None** / **Err**, tranche de **str** coupant un caractère.
- Voir la pile d'appels : variable d'environnement **RUST_BACKTRACE=1**.

L'OPÉRATEUR ?

```
use std::error::Error;
use std::fs;

fn lire_nombre(chemin: &str) -> Result<i64, Box<dyn Error>> {
    let texte = fs::read_to_string(chemin)?; // io::Error
    let n: i64 = texte.trim().parse()?;      // ParseIntError
    Ok(n * 2)
}

fn main() -> Result<(), Box<dyn Error>> {
    let n = lire_nombre("nombre.txt");
    println!("{}", n);
    Ok(())
}
```

- **x?** : si **Ok(v)**, donne **v** ; si **Err(e)**, renvoie aussitôt **Err(e.into())**.
- Fonctionne aussi sur **Option** dans une fonction qui renvoie **Option**.
- Seulement dans une fonction qui renvoie **Result** ou **Option** (sinon **E0277**).
- **main** peut renvoyer **Result** : une erreur y est affichée (format **Debug**) et le programme se termine avec un code d'échec.

EXTRAIRE OU TRANSFORMER : OPTION ET RESULT

match · if let · let else	traiter chaque cas explicitement
?	propager à l'appelant
.unwrap()	❗ la valeur, ou panique (prototypes, tests)
.expect("raison")	idem, avec l'hypothèse expliquée
.unwrap_or(v)	la valeur, sinon v
.unwrap_or_else(...)	sinon calculée (seulement si besoin)
.unwrap_or_default()	sinon 0 , "", vide...
.is_some() · .is_ok()	tests (et is_none , is_err)
.is_some_and(x ...)	présente et conforme
.map(x ...)	transformer la valeur si présente
.and_then(x ...)	enchaîner une opération faillible
.or(b) · .or_else(...)	solution de repli
.filter(x ...)	Option : garder si la condition est vraie
.map_err(e ...)	Result : transformer l'erreur
.ok_or(e) · .ok_or_else(e)	Option → Result
.ok() · .err()	Result → Option
.as_deref()	&Option<String> → Option<&str>
.take()	extraire, en laissant None
.copied() · .cloned()	Option<T> → Option<T>

Une suite de **Result** → un seul : **.collect::<Result<Vec<_>, _>>()** (arrêt à la première erreur).

DÉFINIR SON TYPE D'ERREUR

```
use std::{fmt, io};

#[derive(Debug)]
enum ErreurConfig {
    Absente(String),
    Syntaxe(usize),
    Io(io::Error),
}

impl fmt::Display for ErreurConfig {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            Self::Absente(c) => write!(f, "clé {c} absente"),
            Self::Syntaxe(n) => write!(f, "erreur ligne {n}"),
            Self::Io(e) => write!(f, "entrée-sortie : {e}"),
        }
    }
}

impl std::error::Error for ErreurConfig {}

impl From<io::Error> for ErreurConfig { // pour ?
    fn from(e: io::Error) -> Self { Self::Io(e) }
}
```

- Bibliothèque : type d'erreur précis (**enum**), pour que l'appelant distingue les cas.
- Application : **Box<dyn Error>** suffit souvent.
- Crates tierces répandues : **thiserror** (écrit ce code) et **anyhow** (erreurs d'application, contexte) — à évaluer comme tout code externe.

FICHIERS ET SYSTÈME

fs::read_to_string(p)?	tout le fichier → String
fs::write(p, contenu)?	créer ou écraser
fs::read(p)?	octets bruts → Vec<u8>
File::open(p)? · File::create(p)?	ouvrir en lecture · créer (écrase)
OpenOptions::new().append(true).create(true).open(p)?	ajouter à la fin
fs::create_dir_all(p)?	créer les dossiers manquants
fs::read_dir(p)?	lister un dossier
Path::new(p).exists()	chemins portables : Path , PathBuf
std::env::args()	arguments ; le premier : le programme
std::env::var("NOM")	variable d'environnement → Result
std::process::exit(1)	quitter avec un code de retour

```
use std::fs::File;
use std::io::{self, BufRead, BufReader, Write};

for ligne in BufReader::new(File::open("a.txt")?).lines() {
    let ligne = ligne?; // chaque ligne : Result
    println!("{}", ligne);
}

let mut sortie = File::create("rapport.txt");
writeln!(sortie, "total : {}", 42);
```

LIRE LE CLAVIER

```
println!("Votre âge : ");
io::stdout().flush()?; // afficher aussitôt
let mut saisie = String::new();
io::stdin().read_line(&mut saisie)?;
let age: u32 = saisie.trim().parse()?; // trim : retire \n

write!, writeln! et flush exigent le trait Write : use std::io::Write.
```

6 · Collections, texte et itérateurs

VEC<T> : TABLEAU DYNAMIQUE

<code>Vec::new()</code> · <code>vec![1, 2]</code> · <code>vec![0; n]</code>	créer
<code>Vec::with_capacity(n)</code>	créer en réservant la place
<code>v.push(x)</code> · <code>v.pop()</code>	ajouter à la fin · retirer (→ Option)
<code>v[i]</code> · <code>v.get(i)</code>	accès : panique hors limites · → Option <&T>
<code>v.first()</code> · <code>v.last()</code>	premier · dernier → Option
<code>v.len()</code> · <code>v.is_empty()</code>	taille · vide ?
<code>v.insert(i, x)</code> · <code>v.remove(i)</code>	au rang <i>i</i> (décale la suite)
<code>v.swap_remove(i)</code>	retirer sans décaler (le dernier prend la place)
<code>v.contains(&x)</code>	présence
<code>v.sort()</code> · <code>v.sort_unstable()</code>	trier (éléments Ord)
<code>v.sort_by_key(p p.age)</code>	trier selon une clé
<code>v.sort_by(a, b b.cmp(a))</code>	ordre libre (ici décroissant)
<code>v.dedup()</code>	retirer les doublons consécutifs
<code>v.retain(x *x > 0)</code>	ne garder que ce qui vérifie la condition
<code>v.extend(autre)</code>	ajouter tous les éléments
<code>v.truncate(n)</code> · <code>v.clear()</code>	raccourcir · vider
<code>v.reverse()</code> · <code>v.swap(i, j)</code>	inverser · échanger, sur place
<code>v.binary_search(&x)</code>	vecteur trié → Ok(rang) ou Err(rang d'insertion)
<code>v.chunks(3)</code> · <code>v.windows(2)</code>	paquets de 3 · fenêtres glissantes
<code>v.split_at(i)</code>	deux tranches
<code>v.join(", ")</code>	Vec de textes → un texte

STRING ET &STR

<code>String::new()</code> · <code>String::from("a")</code>	créer ; ou <code>"a".to_string()</code>
<code>s.push('c')</code> · <code>s.push_str("abc")</code>	ajouter
<code>s1 + &s2</code> · <code>format!("{}", s1 {s2})</code>	concaténer (+ consomme <i>s1</i>)
<code>s.len()</code>	❗ longueur en octets : <code>"é".len()</code> → 2
<code>s.chars().count()</code>	nombre de caractères
<code>s.chars()</code> · <code>s.bytes()</code>	parcourir caractères · octets
<code>s.trim()</code>	sans espaces ni <code>\n</code> aux extrémités
<code>s.split(',')</code> · <code>s.lines()</code>	découper (itérateurs)
<code>s.split_whitespace()</code>	mots séparés par des blancs
<code>s.split_once('=')</code>	→ Option <&str, &str>
<code>s.contains("ab")</code>	aussi starts_with , ends_with
<code>s.find("ab")</code>	position en octets → Option <usize>
<code>s.replace("a", "b")</code>	remplacer → nouvelle String
<code>s.to_uppercase()</code>	casse Unicode (to_lowercase)
<code>s.repeat(3)</code>	répéter
<code>s.parse::<f64>()</code>	convertir → Result
<code>&s[0..3]</code>	❗ par octets : panique si un caractère est coupé
❗ <code>s[0]</code> est refusé : en UTF-8, un caractère occupe de 1 à 4 octets. Premiers caractères : <code>s.chars().take(n).collect::<String>()</code> .	

AUTRES COLLECTIONS (STD::COLLECTIONS)

VecDeque <T>	file à deux bouts : push_front , pop_front
HashMap <K, V>	dictionnaire, ordre quelconque
BTreeMap <K, V>	dictionnaire trié par clé
HashSet <T> · BTreeSet <T>	ensemble sans doublon : union , intersection
BinaryHeap <T>	file de priorité, plus grand en tête ; Reverse(x) : plus petit
Clés Hash ... : Eq + Hash ; clés BTree ... : Ord .	

HASHMAP

```
use std::collections::HashMap;

let mut notes: HashMap<String, u32> = HashMap::new();
notes.insert("Anne".to_string(), 15);
if let Some(n) = notes.get("Anne") { println!("{}", n); }

let mut compte = HashMap::new(); // compter les mots
for mot in texte.split_whitespace() {
    *compte.entry(mot).or_insert(0) += 1;
}
for (cle, val) in &notes { println!("{cle} : {val}"); }

m.insert(k, v) // ajoute ou remplace → ancienne valeur (Option)

m.get(&k) · m.get_mut(&k) // → Option
m.contains_key(&k) · m.remove(&k) // présence · retrait

m.entry(k).or_insert(v) // accès, en créant si absent
m.entry(k).or_default() // idem, valeur par défaut
m.keys() · m.values() // itérateurs
```

❗ Ordre de parcours imprévisible, variable d'une exécution à l'autre : **BTreeMap**, ou trier, si l'ordre compte.

ÉCRIRE SON PROPRE ITÉRATEUR

```
struct Compte { n: u32 }
impl Iterator for Compte {
    type Item = u32; // type produit
    fn next(&mut self) -> Option<u32> {
        self.n += 1;
        if self.n <= 3 { Some(self.n) } else { None }
    }
}
// Compte { n: 0 }.collect::<Vec<u32>()>() → [1, 2, 3]
```

Seul **next** est à écrire : **map**, **filter**, **sum**... sont fournis par le trait.

ITÉRATEURS

```
let v = vec![3, 1, 4, 1, 5, 9, 2, 6];
let doubles: Vec<i32> = v.iter()
    .filter(|x| *x % 2 == 0) // 4, 2, 6
    .map(|x| x * 2)
    .collect(); // [8, 4, 12]
let total: i32 = v.iter().sum(); // 31
```

CRÉER

<code>v.iter()</code> · <code>v.iter_mut()</code>	éléments &T · &mut T
<code>v.into_iter()</code>	consomme la collection → T
ADAPTER (PARESSEUX)	
<code>map(f)</code> · <code>filter(p)</code>	transformer · garder si
<code>filter_map(f)</code>	les deux ; <i>f</i> renvoie une Option
<code>enumerate()</code> · <code>zip(b)</code>	→ (rang , <i>x</i>) · paires (<i>a</i> , <i>b</i>)
<code>chain(b)</code> · <code>rev()</code>	à la suite · à l'envers
<code>skip(n)</code> · <code>take(n)</code>	sauter · prendre <i>n</i> éléments
<code>skip_while(p)</code> · <code>take_while(p)</code>	tant que la condition est vraie
<code>step_by(n)</code>	un élément sur <i>n</i>
<code>flat_map(f)</code> · <code>flatten()</code>	aplatir
<code>copied()</code> · <code>cloned()</code>	&T → T

CONSOMMER

<code>collect()</code>	→ Vec , String , HashMap ... (type à préciser)
<code>sum()</code> · <code>product()</code> · <code>count()</code>	somme · produit · nombre
<code>min()</code> · <code>max()</code>	→ Option ; aussi min_by_key(f) , max_by_key(f)
<code>fold(init, f)</code> · <code>reduce(f)</code>	accumuler
<code>any(p)</code> · <code>all(p)</code>	au moins un · tous
<code>find(p)</code> · <code>position(p)</code>	premier trouvé · son rang
<code>partition(p)</code>	→ deux collections
<code>for_each(f)</code>	action sur chaque élément
❗ Paresseux : sans consommateur (collect , sum , for ...), rien n'est calculé.	

7 · Projet : Cargo, modules, tests, documentation

STRUCTURE D'UN PROJET

```
mon_projet/
+-- Cargo.toml      manifeste : nom, dépendances
+-- Cargo.lock      versions exactes retenues (génééré)
+-- src/
|   +-- main.rs      exécutable : fn main
|   +-- lib.rs        bibliothèque (facultatif)
|   +-- config.rs     module config
|   +-- reseau.rs     module reseau...
|   +-- reseau/
|       +-- client.rs sous-module reseau::client
+-- tests/           tests d'intégration
+-- exemples/        cargo run --example nom
+-- benches/         mesures de performance
+-- target/          produits de compilation (hors Git)
```

CARGO.TOML

```
[package]
name = "mon_projet"
version = "0.1.0"
edition = "2024"

[dependencies]
regex = "1" # >= 1.0.0 et < 2.0.0
serde = { version = "1", features = ["derive"] }
outil = { path = "../outil" } # dépendance locale

[dev-dependencies] # pour les tests seulement
proptest = "1"

[profile.release]
lto = true # optimisation plus poussée
```

Versions : "1.2" accepte toute version compatible (≥ 1.2.0 et < 2.0.0) ;
"=1.2.3" : cette version exacte.

COMMANDES CARGO

cargo new <i>nom</i> · cargo init	créer un projet · dans le dossier courant
cargo add <i>regex</i> · cargo remove <i>regex</i>	ajouter · retirer une dépendance
cargo add <i>serde</i> -F <i>derive</i>	avec une fonctionnalité (--features)
cargo build · --release	compiler : debug · optimisé
cargo run -- <i>arg1 arg2</i>	lancer avec des arguments
cargo test <i>motif</i>	tests dont le nom contient le motif
cargo test -- --nocapture	montrer les println! des tests
cargo doc --open	documentation du projet et des dépendances
cargo update	actualiser Cargo.lock (dans les limites de Cargo.toml)
cargo tree	arbre des dépendances
cargo install <i>outil</i>	installer un programme depuis crates.io
cargo publish	❗ publie sur crates.io ; une version publiée ne se remplace pas

❗ Une dépendance est du code tiers, exécuté dès la compilation (**build.rs**, macros) : vérifier auteur, licence, activité, avis de sécurité (rustsec.org ; outil tiers **cargo audit**).

ATTRIBUTS UTILES

#[derive(Debug, Clone)]	implémentations automatiques
#[allow(dead_code)]	taire un avertissement
#[cfg(test)]	compilation conditionnelle (aussi target_os = "windows"...)
#[must_use]	avertir si le résultat est ignoré
#![...]	en tête de fichier : vaut pour tout le module

MODULES ET CHEMINS

```
// src/main.rs
mod geometrie; // charge src/geometrie.rs
use geometrie::{distance, Point};
use std::io::{self, Write}; // io et io::Write

// src/geometrie.rs
pub struct Point { pub x: f64, pub y: f64 }

pub fn distance(a: &Point, b: &Point) -> f64 {
    (a.x - b.x).hypot(a.y - b.y)
}

mod m; // déclarer un module : fichier m.rs (ou m/mod.rs)

mod m { ... } // module écrit sur place
use a::b::C; // raccourcir un chemin
use a::b::C as D; // importer sous un autre nom
use a::b::*; // tout importer (à éviter, sauf tests)
crate:: · super:: · // racine de la crate · module parent · module courant
self::
pub use a::C; // réexporter : façade publique
```

Crate : unité de compilation (un exécutable ou une bibliothèque). **Paquet** (package) : un **Cargo.toml** et ses crates.

TESTS

```
pub fn diviser(a: i32, b: i32) -> i32 {
    if b == 0 { panic!("division par zéro"); }
    a / b
}

#[cfg(test)] // compilé pour cargo test
mod tests {
    use super::*;

    #[test]
    fn quotient() {
        assert_eq!(diviser(7, 2), 3);
        assert!(diviser(-6, 3) < 0, "négatif attendu");
    }

    #[test]
    #[should_panic(expected = "division par zéro")]
    fn par_zero() { diviser(1, 0); }

    #[test]
    fn lecture() -> Result<(), std::num::ParseIntError> {
        assert_eq!("42".parse::<i32>()?, 42);
        Ok(())
    }
}

assert_eq!(a, b) · // égalité · différence ; message facultatif en plus
assert_ne!
#[should_panic] // le test doit paniquer
#[ignore] // ignoré, sauf cargo test -- --ignored
tests/*.rs // tests d'intégration : API publique seulement
```

DOCUMENTER

```
/// Moyenne d'une tranche ; `None` si elle est vide.
///
/// ...
/// assert_eq!(mon_projet::moyenne(&[2.0, 4.0]), Some(3.0));
/// ...
pub fn moyenne(v: &[f64]) -> Option<f64> {
    if v.is_empty() { return None; }
    Some(v.iter().sum::<f64>() / v.len() as f64)
}

/// documente l'élément suivant (Markdown) ; ///! le module entier. Les blocs
de code de la documentation sont exécutés par cargo test (bibliothèques).
```

8 · Concurrency, idiomes, pièges, ressources

THREADS

```
use std::sync::{mpsc, Arc, Mutex};
use std::thread;

let h = thread::spawn(|| 6 * 7);
let r = h.join().unwrap(); // 42

let compteur = Arc::new(Mutex::new(0));
let mut poignées = vec![];
for _ in 0..4 {
    let c = Arc::clone(&compteur);
    poignées.push(thread::spawn(move || {
        *c.lock().unwrap() += 1;
    }));
}
for p in poignées { p.join().unwrap(); }
// *compteur.lock().unwrap() vaut 4

let (tx, rx) = mpsc::channel(); // canal
thread::spawn(move || tx.send("message").unwrap());
println!("{}", rx.recv().unwrap());

let v = vec![1, 2, 3];
thread::scope(|s| { // threads bornés :
    s.spawn(|| println!("{v:?}")); // emprunt permis
});
```

- **Send** : transférable à un autre thread · **Sync** : partageable par référence. **Rc** ou **RefCell** entre threads : refusé (E0277).
- **lock()** renvoie **Err** si un thread a paniqué en tenant le verrou (verrou « empoisonné »).
- **!** Le verrou reste pris jusqu'à la fin de la portée du garde : sections courtes. Deux verrous pris dans des ordres différents → interblocage, que le compilateur ne détecte pas.

ASYNCHRONE : ASYNC / AWAIT

- **async fn** renvoie un **futur** : rien ne s'exécute avant **.await**.
- La bibliothèque standard ne fournit pas d'exécuteur : il vient d'une crate tierce, le plus souvent **tokio**.
- Pour de nombreuses attentes d'entrées-sorties (réseau, serveurs). Pour du calcul : **threads**.

```
#[tokio::main]
async fn main() {
    let (a, b) = tokio::join!(tache_a(), tache_b());
}
```

MACROS ET UNSAFE

```
macro_rules! carre {
    ($x:expr) => { $x * $x };
}
let n = carre!(3 + 1); // 16 : $x reste groupé
```

Un bloc **unsafe { }** autorise **cinq** opérations invérifiables : déréférencer un pointeur brut, appeler une fonction **unsafe** (dont le code C), lire ou modifier un **static mut**, implémenter un trait **unsafe**, lire un champ d'**union**. Les autres règles restent actives. Un programme d'application n'en a presque jamais besoin.

BOÎTE À OUTILS DE LA BIBLIOTHÈQUE STANDARD

mem::swap(&mut a, &mut b)	échanger deux valeurs
mem::take(&mut v)	extraire, en laissant la valeur par défaut
mem::replace(&mut v, n)	remplacer, en récupérant l'ancienne
a.cmp(&b)	→ Ordering::Less, Equal ou Greater
Instant::now() · t.elapsed()	mesurer une durée (std::time)
Command::new("git").arg("status").output()?	lancer un programme (std::process)
c.is_alphabetic() · c.is_ascii_digit()	nature d'un caractère
c.to_digit(10) · char::from_digit(7, 10)	chiffre ↔ caractère

IDIOMES

- Constructeur **new()** et **impl Default; impl From<X> for Y** pour convertir (utilisé par ?).
- Paramètres **&str** et **&[T]** ; valeurs renvoyées possédées.
- Itérateurs plutôt que boucles sur des indices.
- « Newtype » **struct Euros(f64)** ; ne plus confondre deux unités.
- Rendre les états impossibles irréprésentables : une **enum** plutôt que des booléens combinés.
- **Option** pour l'absence, jamais de valeur sentinelle (-1, "").

PIÈGES FRÉQUENTS

.clone() pour faire taire le compilateur	revoir qui possède quoi ; emprunter
.unwrap() partout	?, expect , traitement explicite
s.len() pris pour des caractères	s.chars().count()
&s[.n] sur un texte accentué	panique : s.chars().take(n)
dépassement d'entier silencieux en --release	checked_add... ; tester en debug
v.sort() sur des f64	refusé : trier avec total_cmp (page 4)
itérateur sans consommateur	rien n'est calculé
modifier une collection en la parcourant	refusé : retain , ou collecter d'abord
performances mesurées en debug	toujours --release

OUTILS AUTOUR DU COMPILATEUR

- **rust-analyzer** : extension d'éditeur (VS Code...) — complétion, types affichés, erreurs en direct. **cargo fmt** et **cargo clippy** avant chaque commit.
- **cargo clippy --fix** : appliquer les corrections proposées.

DE PYTHON À RUST

PYTHON	RUST
None	Option<T> : None, Some(x)
try / except	Result<T, E>, ?, match
list · dict · set	Vec · HashMap · HashSet
for i in range(n)	for i in 0..n
len(s)	s.chars().count() (s.len() : octets)
f"{x}"	format!("{}", x)
class · héritage	struct + impl · traits et composition
lambda x: x + 1	 x x + 1
pip · pytest	cargo add · cargo test

GLOSSAIRE FRANÇAIS – ANGLAIS

possession <i>ownership</i>	emprunt <i>borrow</i>
vérificateur d'emprunts <i>borrow checker</i>	
durée de vie <i>lifetime</i>	libération <i>drop</i>
tranche <i>slice</i>	fermeture <i>closure</i>
filtrage par motif <i>pattern matching</i>	déplacement <i>move</i>
pointeur intelligent <i>smart pointer</i>	masquage <i>shadowing</i>
mutabilité intérieure <i>interior mutability</i>	

POUR ALLER PLUS LOIN

- *Le langage de programmation Rust* (traduction du « Book ») : jims-kapit.github.io/rust-book-fr
- *Comprehensive Rust* (Google), en français : google.github.io/comprehensive-rust/fr
- *Rust by Example* : doc.rust-lang.org/rust-by-example
- *Rustlings*, exercices guidés par le compilateur : github.com/rust-lang/rustlings
- Bibliothèque standard : doc.rust-lang.org/std · paquets : crates.io, docs.rs
- Essayer sans rien installer : play.rust-lang.org